ÉNONCE TRAVAUX PRATIQUE – SEMAINE 1

Introduction au JavaScript

Thèmes

- Les structures de contrôle
- Affichage dans la console
- Prendre en main PhpStorm

Présentation

Chaque semaine, vous aurez une séance de Travaux Pratiques (TP) sur le Développement d'Interface Web. Le sujet contiendra plusieurs exercices qu'il est préférable de traiter dans l'ordre. Dans la majorité des cas, la durée de la séance sera suffisante pour « terminer » les exercices. La bonne pratique de finaliser l'ensemble du sujet, pour bien assimiler les savoirs de la semaine courante avant la prochaine. Évidemment, le travail personnel en dehors des séances pourra vous aider.

En plus de la découverte du langage JavaScript, il est important de prendre en main PhpStorm, qui nous servira tout au long de la ressource, afin d'être le plus efficace possible.

Exercice 1 : On prend ses marques

Pour cet exercice, nous allons améliorer votre prise en main de **PhpStorm**. À la fois en utilisant des raccourcis, mais aussi en découvrant les fonctionnalités qui vous aiderons dans vos travaux de développement.

À faire :

- Récupérer un projet existant depuis GitHub, dans le menu **file**, choisir **new**, puis **Project from version control**. Dans la fenêtre qui s'ouvre, vous saisirez le champ URL le lien **https://github.com/laurentgiustignano/R2.02-TP1.1**. Dans le champ **Directory**, vous vous placerez dans votre répertoire de TP sur **Z :**.
- Ouvrez l'unique fichier **index.html**. Il contient un titre et une portion de script JS. En cliquant sur l'icône de **Firefox** apparaissant en haut à droite, vous ouvrez la page HTML en utilisant le serveur web intégré de **PhpStorm**.
- Faites un clic-droit dans la page HTML et choisissez **Inspecter**, puis l'onglet **console**. Confirmez que le message présent correspond au contenu de **index.html**.
- Injecter du code JS directement dans les balises **<script>** n'est pas une bonne pratique, et nécessite d'être présent dans un fichier séparé. Sélectionnez le code JS à l'intérieur des balises, puis avec un clic-droit choisir **Refactor,** puis **Extract Include File**. Nommez le nouveau fichier **message.js** dans la fenêtre contextuelle, puis validez. Appréciez la modification de l'élément HTML **script** et vérifiez que le fonctionnement n'est pas altéré.

Exercice 2 – Tu as quel âge ?

Dans le même projet, vous allez créer un nouveau script **ageLegal.js** qui permettra de déterminer si un utilisateur est majeur ou mineur après avoir saisi son année de naissance. Cette saisie s'effectue avec **prompt()**. Le message de retour doit être dans la console (*cf : image ci-dessous*), et vous devez alterner l'utilisation de la concaténation de chaine pour insérer l'âge de l'utilisateur et l'usage des **backtick**.

Exemple de sortie :

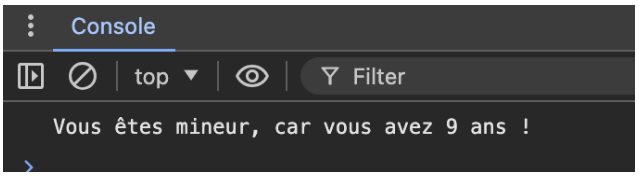

Figure 1 - Affichage de sortie dans la console après avoir saisi 2015

Exercice 3 – Elle est triste ma console ?

Dans le même projet, dupliquez la logique du dernier exercice dans un nouveau script **ageCouleur.js**. L'affichage de majeur ou mineur devra être mis en évidence avec du CSS. Le maximum d'informations devra être factorisé. La couleur du texte est verte ou rouge, respectivement pour majeur ou mineur. Le font sera commun pour les deux cas à blanc et la taille sera doublée comme dans l'exemple ci-dessous.

Exemple de sortie :

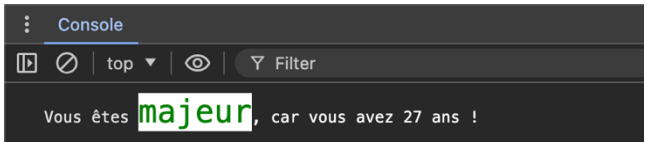

Figure 2 - Affichage de sortie dans la console après avoir saisi 1997

Exercice 4 – On table sur combien ?

- Récupérer un projet existant depuis l'URL : **https://github.com/laurentgiustignano/R2.02-TP1.4**. Le fichier **users.js** contient une seule déclaration contenant des données générées aléatoirement. Déterminez le type **users** et son contenu.
- Effectuez un affichage dans la console de votre navigateur de l'élément **users** pour confirmer le contenu. À l'aide d'une boucle, affichez séparément les éléments de **users**. Enfin, ajoutez dans le corps de la boucle le mot-clé **debugger** pour insérer un point d'arrêt à l'exécution. Maintenant, vous pouvez exécuter le script en pas-à-pas.
- Modifiez le fichier **users.js** pour ajouter à chaque numéro de téléphone l'indicatif de la Norvège sous la forme « 47- ». Puis, effectuez l'affichage de l'information **username** , du téléphone et la ville de résidence de chaque utilisateur.
- L'usage de **console.table()** à la place de **console.log()** permet présenter le contenu d'un tableau directement sans boucle. En consultant la documentation en ligne (**https://developer.mozilla.org/en-US/docs/Web/API/console/table_static**), obtenez un affichage contenant uniquement les informations **username** et **website**.

Exercice 5 – On renverse la table !

Dans cet exercice, vous allez écrire un programme qui calcule la transposée d'une matrice. Le code sera placé dans un fichier matrice.js, et un fichier matrice.html permettra de charger et d'exécuter ce script dans le navigateur. Vous afficherez dans la console la matrice originale ainsi que sa transposée dans la console.

*Rappel : La **transposée d'une matrice** est une nouvelle matrice obtenue en échangeant ses lignes et ses colonnes. Autrement dit, les éléments qui se trouvent sur une ligne dans la matrice d'origine se retrouvent dans une colonne dans la matrice transposée. Par exemple, si on a une matrice avec 2 lignes et 3 colonnes, sa transposée aura 3 lignes et 2 colonnes*

À faire :

- Créer les fichiers **matrice.js** et **matrice.html**, puis établir le lien entre les fichiers.
- Déclarer une variable **const matriceOrigine** contenant 12 valeurs numériques sur 3 lignes et 4 colonnes.
- À l'aide de boucles, remplissez un tableau préalablement créé avec les valeurs transposées depuis **matriceOrigine**. Affichez l'ensemble dans la console.

Bon courage…